

Laboratorio 5: Clasificación de tweets usando minería de texto

Informe de avance

Fecha de entrega del avance: 27 de agosto de 2026, 17:20

1. Introducción

Este informe documenta el avance del Laboratorio 5, cuyo objetivo final es construir un modelo de clasificación que determine si un tweet se refiere a un desastre real (target = 1) o no (target = 0). Se utiliza el dataset "Natural Language Processing with Disaster Tweets" de Kaggle. En esta primera entrega se presentan: la descripción de los datos, el preprocesamiento de texto realizado, el análisis de unigramas y bigramas, y la descripción de un modelo preliminar de clasificación.

2. Descripción del dataset

El archivo train.csv contiene 7,613 tweets con las siguientes columnas: id (identificador), keyword (palabra clave, puede estar vacía), location (ubicación de origen, puede estar vacía), text (texto del tweet) y target (1 = desastre real, 0 = no desastre).

Variable	Nulos	% Nulos	Observación
keyword	61	0.80%	221 valores distintos; se puede imputar o usar como feature adicional
location	2,533	33.27%	3,341 valores distintos, muy ruidosa (texto libre); requiere limpieza fuerte si se usa
text	0	0.00%	Sin valores nulos
target	0	0.00%	Variable objetivo, sin nulos

Tabla 1. Resumen de valores nulos por variable.

La distribución de la variable objetivo muestra un ligero desbalance: 4,342 tweets (57.0%) no corresponden a un desastre real y 3,271 (43.0%) sí. No es un desbalance severo, por lo que no se considera necesario aplicar técnicas de balanceo (SMOTE, undersampling) por ahora, aunque se evaluará el desempeño por clase con precision/recall en lugar de depender únicamente del accuracy.

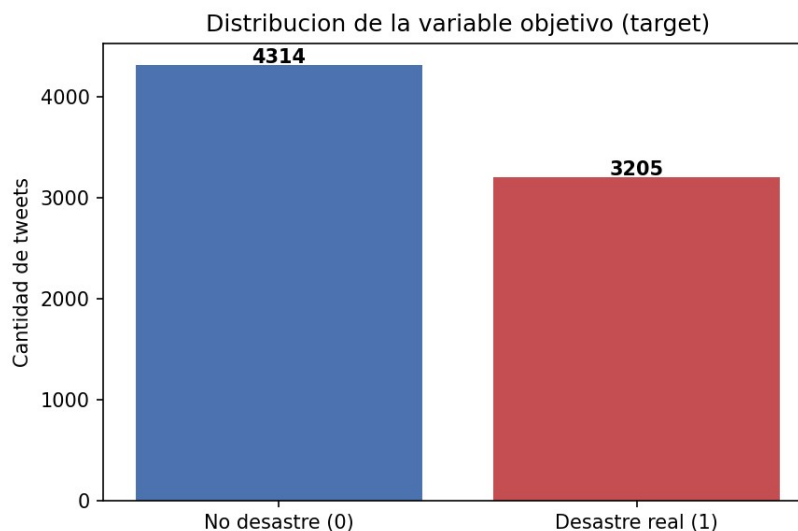


Figura 1. Distribución de la variable objetivo (target).

Se identificaron además 92 tweets duplicados exactos (mismo texto y mismo target), los cuales fueron eliminados para evitar sesgo en el conteo de frecuencias y en el entrenamiento del modelo. La longitud promedio de un tweet es de aproximadamente 101 caracteres y 15 palabras, consistente con el límite histórico de Twitter de 140 caracteres.

3. Limpieza y preprocesamiento de datos

Se aplicó una función de limpieza (`clean_text`) en Python usando expresiones regulares y la librería NLTK. Los paquetes utilizados fueron: `pandas` para manejo de datos, `re` para expresiones regulares, y `nltk.corpus.stopwords` para la eliminación de palabras vacías en inglés. Las tareas realizadas, en orden, fueron:

- Corrección de "mojibake": el texto original de Twitter contenía secuencias como `\x89ÛÒ` producto de una mala codificación de caracteres especiales (comillas curvas, elipsis). Se eliminaron los caracteres no-ASCII, ya que inicialmente aparecían como palabras "fantasma" (ej. "û_") contaminando el top de frecuencias.
- Conversión de todo el texto a minúsculas, para que "Fire" y "fire" se traten como el mismo token.
- Eliminación de URLs (`http`, `https`, `www`) mediante expresiones regulares, ya que no aportan información semántica.
- Eliminación de menciones (`@usuario`).
- Eliminación del símbolo "#" de los hashtags, pero conservando la palabra (ej. "#earthquake" → "earthquake"), ya que suele contener información temática relevante.
- Eliminación de emojis/emoticones mediante rangos Unicode.
- Eliminación de entidades HTML (ej. `&`).
- Eliminación de signos de puntuación y guiones bajos residuales de la limpieza de mojibake.
- Eliminación de números, con una excepción deliberada: se conserva el token "911", porque en este dominio (clasificación de desastres) suele ser una señal semánticamente relevante (llamadas de emergencia, ej. "call 911").
- Eliminación de stopwords en inglés (artículos, preposiciones, conjunciones) usando la lista de NLTK, exceptuando las negaciones ("no", "not", "nor"), que se conservan pensando en el análisis de sentimiento del punto 8 del laboratorio, donde una negación puede invertir la polaridad de una frase.
- Eliminación de espacios múltiples y tokens de un solo carácter.

A continuación se muestran ejemplos de tweets antes y después del preprocesamiento:

Texto original	Texto limpio
Our Deeds are the Reason of this #earthquake May ALLAH Forgive us all	deeds reason earthquake may allah forgive us
Forest fire near La Ronge Sask. Canada	forest fire near la ronge sask canada
13,000 people receive #wildfires evacuation orders in California	people receive wildfires evacuation orders california

Tabla 2. Ejemplos de texto antes y después de la limpieza.

Tras la limpieza, únicamente 2 tweets quedaron con texto vacío (tweets compuestos solo por URL/menciones); se conservan en el dataset pero se excluyen del análisis de frecuencias y del entrenamiento del modelo.

4. Frecuencia de palabras y n-gramas

Se calcularon las frecuencias de unigramas, bigramas y trigramas por separado para cada clase, usando CountVectorizer de scikit-learn. Esto permite identificar qué palabras y combinaciones de palabras son más características de cada categoría.

4.1 Unigramas más frecuentes

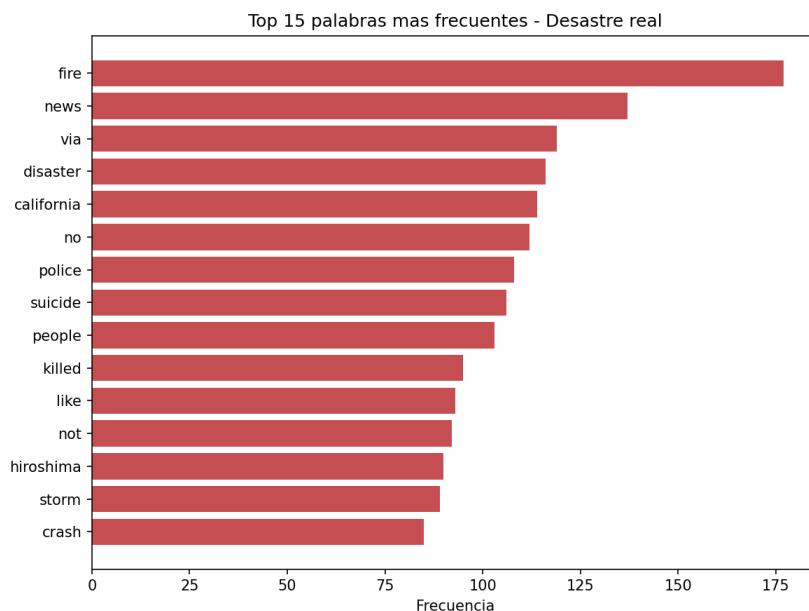


Figura 2. Top 15 palabras más frecuentes en tweets de desastre real.

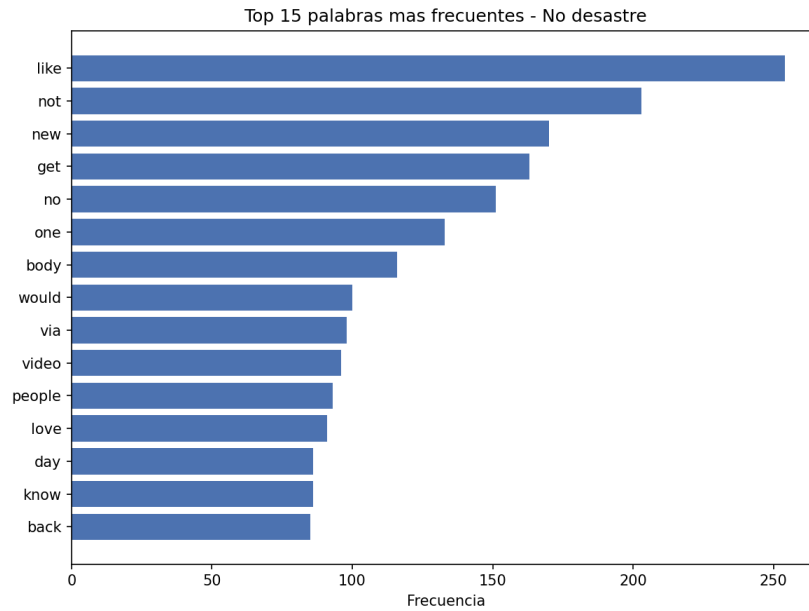


Figura 3. Top 15 palabras más frecuentes en tweets que no son desastre.

En los tweets de desastre real predominan palabras claramente relacionadas con eventos catastróficos: fire, disaster, police, suicide, killed, storm, crash, families, emergency, nuclear. En los tweets que no son desastre predominan palabras de uso cotidiano o coloquial: like, get, love, day, know, body, video, lo que sugiere que muchos de estos tweets usan palabras como "fire", "bomb" o "crash" en sentido figurado (ej. "that concert was fire").

Se identificaron 14 palabras que aparecen en el top 50 de ambas categorías: burning, emergency, fire, get, like, new, news, no, not, one, people, still, via, video. Estas palabras por sí solas son ambiguas para clasificar y su poder discriminante depende del contexto en que aparecen, lo que confirma la utilidad de analizar bigramas y trigramas.

4.2 Bigramas y trigramas: ¿vale la pena analizarlos?

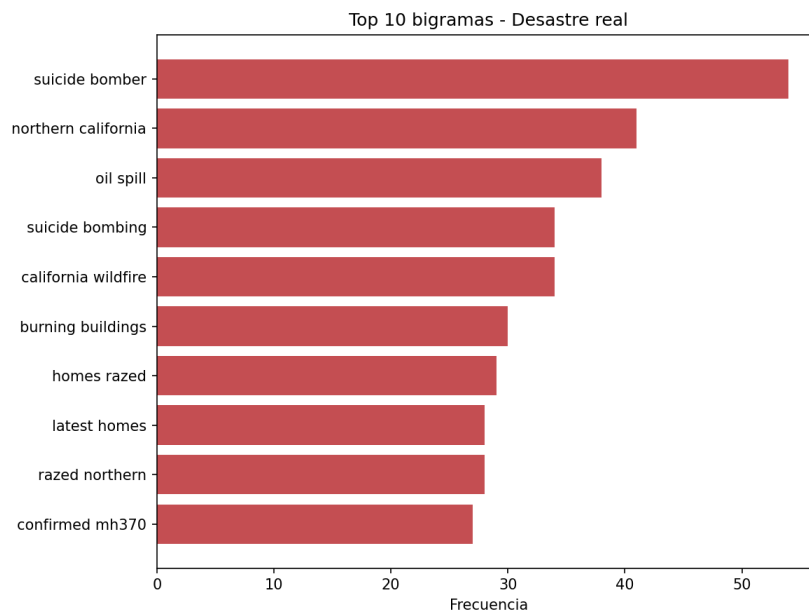


Figura 4. Top 10 bigramas más frecuentes en tweets de desastre real.

Conclusión: se recomienda usar una representación combinada de unigramas + bigramas (ngram_range=(1,2)) para el modelo de clasificación, balanceando cobertura de vocabulario con captura de contexto local, sin sobre-dispersar el vocabulario como podría pasar incluyendo trigramas.

Figura 5. Nube de palabras — tweets de desastre real.

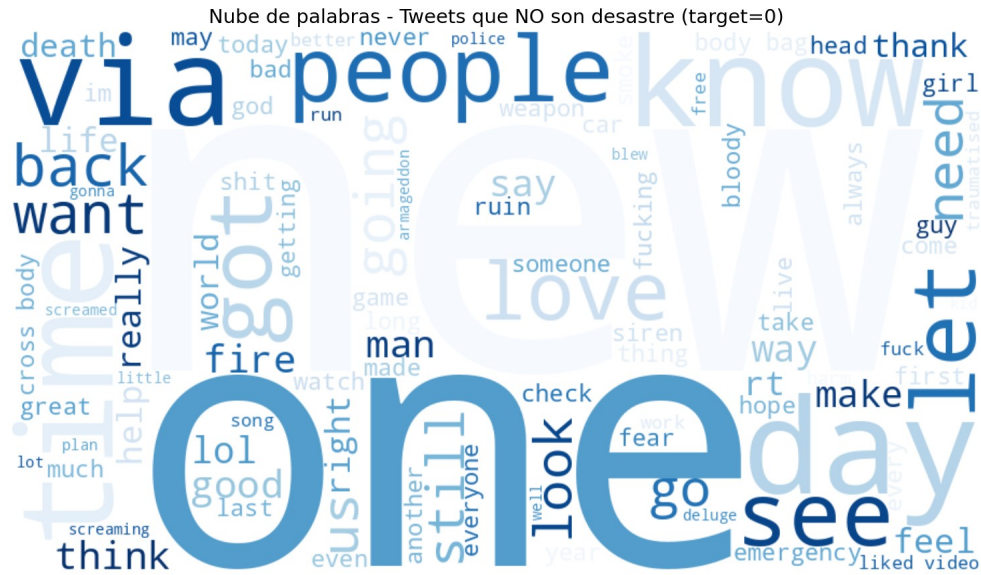


Figura 6. Nube de palabras — tweets que no son desastre.

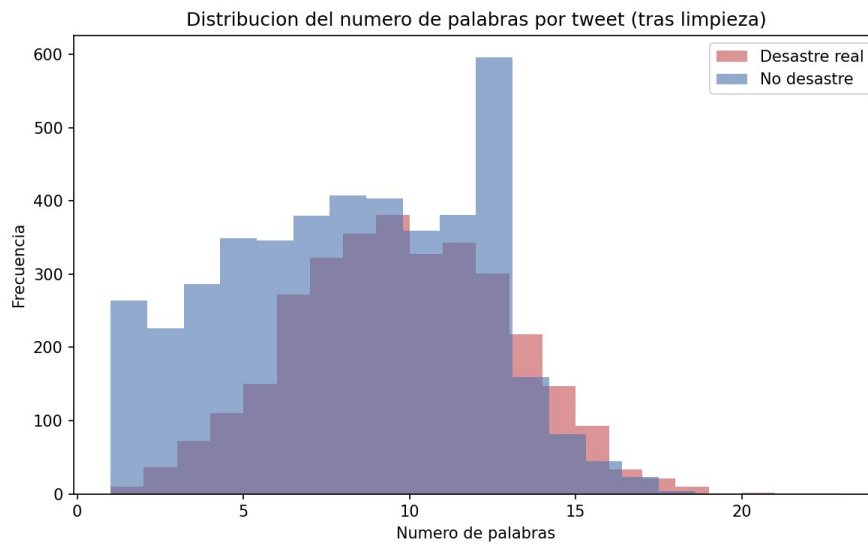


Figura 7. Distribución del número de palabras por tweet, por categoría (tras limpieza).

La distribución de longitud de tweets (en palabras, tras la limpieza) es similar entre ambas categorías, con una media alrededor de 9-10 palabras; no parece ser, por sí sola, una variable fuertemente discriminante, pero se mantendrá disponible como posible feature adicional.

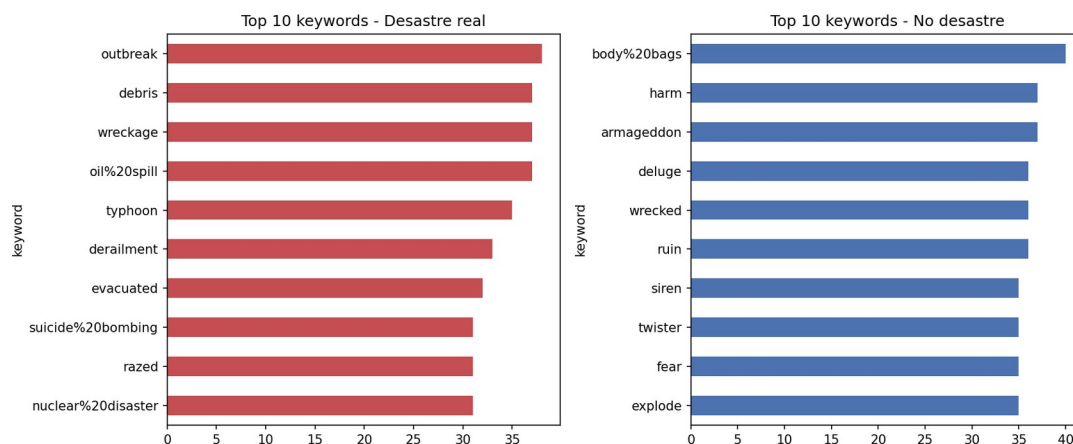


Figura 8. Top 10 valores de la columna 'keyword' original por categoría.

La columna keyword (proporcionada en el dataset original, antes de la limpieza de texto libre) confirma el patrón: keywords como "derailment", "wreckage", "debris" y "outbreak" están fuertemente asociadas a tweets de desastre real, mientras que otras keywords ambiguas (ej. "body bags", "panic") aparecen en ambas categorías dependiendo del uso figurado o literal del término.

6. Modelo preliminar de clasificación

Para abordar el problema de contexto discutido en la sección 4.2, se representó cada tweet limpio mediante TF-IDF con unigramas y bigramas ($\text{ngram_range}=(1,2)$), limitando el vocabulario a las 5,000 características más relevantes para controlar la dimensionalidad. Se separó el conjunto en 80% entrenamiento y 20% prueba, de forma estratificada para conservar la proporción de clases.

Como modelos preliminares (baseline) se entrenaron dos algoritmos ampliamente usados en clasificación de texto: Regresión Logística y Naive Bayes Multinomial. Ambos son rápidos de entrenar y sirven como punto de comparación para los modelos más elaborados que se explorarán en la entrega final (ej. SVM, Random Forest, redes neuronales simples).

Modelo	Accuracy	Precision	Recall	F1-score
Regresión Logística	0.796	0.812	0.679	0.739
Naive Bayes Multinomial	0.802	0.846	0.654	0.738

Tabla 3. Métricas de los modelos preliminares sobre el conjunto de prueba (20%).

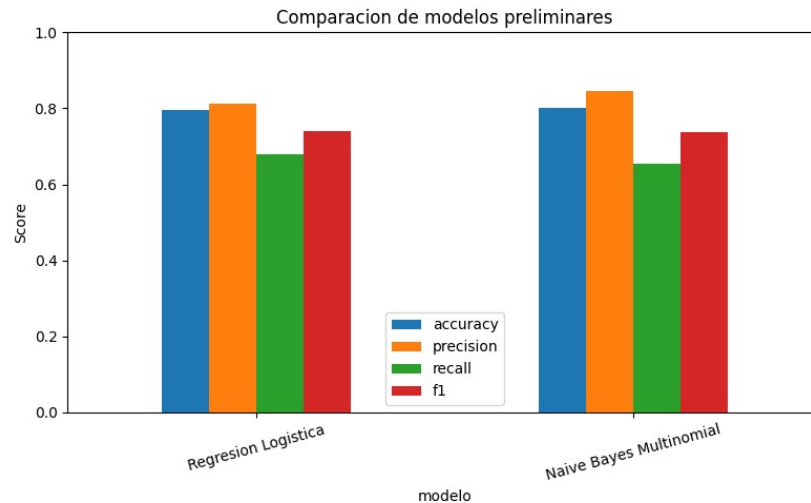


Figura 9. Comparación visual de métricas entre los modelos preliminares.

Ambos modelos alcanzan una exactitud (accuracy) cercana al 80% únicamente con TF-IDF de texto, sin ninguna variable adicional. Naive Bayes obtiene mayor precisión (menos falsos positivos), mientras que Regresión Logística obtiene un recall ligeramente mayor. Se observa que el recall de la clase "desastre real" (0.65–0.68) es menor que el de la clase "no desastre", lo que indica que ambos modelos tienden a perder algunos tweets de desastre real, probablemente aquellos que usan lenguaje menos directo o palabras ambiguas discutidas en la sección 4.

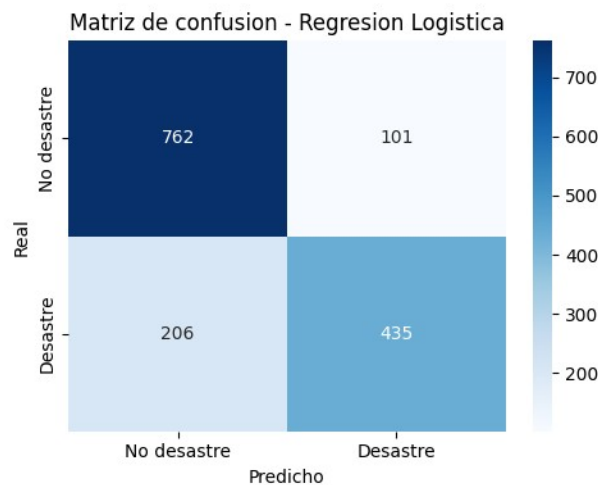


Figura 10. Matriz de confusión — Regresión Logística.

Estos resultados son un punto de partida razonable. En la entrega final se probarán más algoritmos, se realizará ajuste de hiperparámetros, y se incorporará la variable de "negatividad" del tweet (análisis de sentimiento) para evaluar si mejora la capacidad de detectar tweets de desastre real, particularmente el recall de esa clase.

7. Próximos pasos

- Elaborar y comparar más modelos de clasificación (SVM, Random Forest u otros) y seleccionar el mejor con métricas adecuadas.
- Construir la función que reciba un tweet sin preprocesar y devuelva la clasificación (desastre / no desastre).

- Determinar palabras positivas, negativas y neutras por tweet (análisis de sentimiento) usando un léxico o librería como TextBlob/VADER.
- Identificar los 10 tweets más positivos y más negativos, y comparar la negatividad entre categorías.
- Crear la variable de "negatividad" del tweet, incorporarla al modelo y evaluar si mejora los resultados.
- Consolidar todos los hallazgos en el informe final con las referencias de las herramientas externas utilizadas.

8. Referencias y herramientas utilizadas

Librerías de Python: pandas, scikit-learn (CountVectorizer, TfidfVectorizer, LogisticRegression, MultinomialNB, train_test_split, métricas), nltk (stopwords), wordcloud, matplotlib y seaborn (visualización).

Jurafsky, D., & Martin, J. H. (2014). N-Grams. Speech and Language Processing.
<https://lagunita.stanford.edu/c4x/Engineering/CS-224N/asset/slp4.pdf>

Feinerer, I., Hornik, K., & Meyer, D. (2008). Text Mining Infrastructure in R. Journal of Statistical Software, 25(5), 1–54. <https://www.jstatsoft.org/article/view/v025i05>